

Sistema de Detecao de Cyberbullying — Documentacao Tecnica

****Instituto Superior Politecnico da Lunda-Sul — Saurimo, Angola****

PP1 — Engenharia de Software I + Sistemas Operacionais | 2026

Indice

1. [Visao Geral](#1-visao-geral)
2. [Arquitetura do Sistema](#2-arquitetura-do-sistema)
3. [Instalacao e Configuracao](#3-instalacao-e-configuracao)
4. [Guia de Utilizacao](#4-guia-de-utilizacao)
5. [API REST](#5-api-rest)
6. [Motor de Detecao](#6-motor-de-detecao)
7. [Classificador BERT](#7-classificador-bert)
8. [WebSockets e Notificacoes](#8-websockets-e-notificacoes)
9. [Base de Dados](#9-base-de-dados)
10. [Gestao de Utilizadores e Permissoes](#10-gestao-de-utilizadores-e-permissoes)
11. [Testes Automatizados](#11-testes-automatizados)
12. [Empacotamento (EXE)](#12-empacotamento-exe)
13. [Manutencao e Resolucao de Problemas](#13-manutencao-e-resolucao-de-problemas)
14. [Credenciais](#14-credenciais)
15. [Equipa de Desenvolvimento](#15-equipa-de-desenvolvimento)

1. Visao Geral

Sistema multiplataforma para detecao de cyberbullying em texto, desenvolvido para o Instituto Superior Politecnico da Lunda-Sul (Saurimo, Angola). Combina uma interface ****web**** (Flask), uma aplicacao ****desktop**** (Tkinter) e um executavel autonomo (****EXE --onefile****) que partilham a mesma base de dados SQLite e o mesmo motor de detecao enriquecido com dicionario de girias regionais angolanas.

1.1 Funcionalidades Principais

Funcionalidade	Descricao
---	---
Analise de texto em tempo real	Classifica como Critico / Ofen
Dicionario alargado	170+ palavras ofensivas + 80+
Classificacao por niveis	critico, alto, medio, baixo, n
Classificador BERT	distilbert-base-multilingual-c
Alertas inteligentes	Threshold >= 50% de confianca
Notificacoes em tempo real	WebSockets (Flask-SocketIO) co
Interface web e desktop	Mesma BD partilhada, mesma exp
Autenticacao RBAC	Login protegido com senhas has
Auto-save	Toda analise e guardada automa
Exportacao de relatorios	CSV, TXT e PDF
Monitorizacao Facebook	Integracao com Graph API para
Testes automatizados	15 testes pytest (detecao, API
Executavel autonomo	EXE --onefile (PyInstaller) se
Repositorio GitHub	https://github.com/24Anselmo/C

1.2 Tecnologias

Componente	Tecnologia
---	---
Servidor web	Flask 3.0 + Flask-SocketIO
ORM	SQLAlchemy + SQLite (WAL mode)
Frontend	HTML5, CSS3, JavaScript (vanil
Interface desktop	Tkinter (Python)
Motor de detecao	Rule-based (dicionario + pesos

Autenticacao	bcrypt (senhas hasheadas)
WebSockets	Flask-SocketIO + eventlet / th
BERT	transformers + torch (distilbe
PDF	fpdf2
CORS	Flask-CORS
Testes	pytest
Empacotamento	PyInstaller (EXE --onefile / -
Repositorio	Git + GitHub

2. Arquitetura do Sistema

2.1 Estrutura de Diretorios

cyberbullying_detector/

```
+-- app/                                # Aplicacao Web (Flask)
| +-- __init__.py                       # Factory, BD, CORS, SocketIO, migracao
| +-- models.py                         # Modelos SQLAlchemy
| +-- routes.py                         # Endpoints REST + login + PDF
| +-- detector.py                       # Motor de detecao (singleton rule-based)
| +-- bert_classifier.py                # Classificador BERT (distilbert)
| +-- facebook_api.py                  # Monitorizacao Facebook Graph API
| +-- socket_events.py                 # Eventos WebSocket
| +-- static/
| | +-- css/style.css                   # Estilos responsivos
| | +-- js/main.js                      # Logica frontend completa
| | +-- logo.png                       # Logotipo
| +-- templates/
|   +-- index.html                     # Dashboard principal
|   +-- login.html                     # Pagina de autenticacao
+-- offline/                           # Aplicacao Desktop
| +-- desktop_app.py                   # UI Tkinter
| +-- logo.png                         # Logotipo desktop
| +-- CyberbullyingDetector.spec       # Configuracao PyInstaller
| +-- dist/
|   +-- CyberbullyingDetector.exe      # Executavel compilado
+-- data/                              # Dados persistentes
| +-- cyberbullying.db                 # Base de dados SQLite
| +-- local_dictionary.json            # Dicionario local (80+ girias)
| +-- debug.log                       # Log de diagnostico
+-- tests/                             # Testes automatizados
| +-- conftest.py                      # Fixture BD temporario
| +-- test_detector.py                 # Testes do motor de detecao (6)
| +-- test_api.py                      # Testes dos endpoints REST (6)
| +-- test_auth.py                     # Testes de autenticacao (3)
+-- DOCUMENTACAO.html                 # Documentacao (HTML)
+-- DOCUMENTACAO.md                   # Documentacao (Markdown)
+-- README.md                         # Manual rapido
+-- README_SO.md                      # Comandos SO
+-- requirements.txt                   # Dependencias Python
+-- run.py                            # Ponto de entrada web
```

2.2 Fluxo de Funcionamento

```
+-----+
| Base de Dados SQLite |
| data/cyberbullying.db |
+-----+-----+
|
+-----+-----+
|         |         |
v         v         v
```

```
+-----+ +-----+ +-----+
| Desktop |<--->| Servidor |<--->| Web |
| (Tkinter)| | Web Flask | | (Browser)|
| .exe | | :5000 | | localhost |
+-----+ +-----+ +-----+
```

- Desktop e Web partilham o mesmo ficheiro `cyberbullying.db`
- O EXE desktop abre o browser para o utilizador aceder a interface web
- Motor de deteção (`app/detector.py`) e o mesmo para ambas as interfaces
- Classificador BERT (`app/bert\_classifier.py`) e opcional (lazy loading)
- Toda a análise e auto-salva na BD antes de ser exibida
- Notificações em tempo real via WebSockets (Flask-SocketIO)

2.3 Autenticacao e Sessao

- **Web**: sessao Flask armazenada em cookie assinado (`secret\_key`)
- **Desktop**: login verificado localmente via SQLite com bcrypt; `user\_id`, `nome` e `papel` passados ao construtor da app
- **Senhas**: armazenadas com hash bcrypt; migracao automatica de texto plano existente
- **Logout (web)**: `session.clear()` + redirect para `/login`
- **Logout (desktop)**: `root.destroy()` + `main()` (re-inicia ciclo de login)

3. Instalacao e Configuracao

3.1 Requisitos

- Windows 10/11 ou Linux (Ubuntu 22.04+)
- Python 3.10 ou superior
- pip (gestor de pacotes Python)
- Git (opcional, para clonar repositório)

3.2 Obter o Projeto

```
git clone https://github.com/24Anselmo/Cyber123.git
cd cyberbullying_detector
```

3.3 Instalacao de Dependencias

```
pip install -r requirements.txt
```

Conteudo do `requirements.txt`:

```
Flask==3.0.0
Flask-SQLAlchemy==3.1.1
Flask-CORS==4.0.0
Flask-SocketIO==5.6.1
bcrypt==5.0.0
scikit-learn==1.3.2
numpy==1.26.2
pandas==2.1.3
requests==2.31.0
python-dotenv==1.0.0
gunicorn==21.2.0
transformers>=4.30.0
torch>=2.0.0
eventlet>=0.41.0
fpdf2>=2.7.0
```

3.4 Execucão

```
Modo Web (Servidor Flask):
python run.py
```

Acessar: <http://localhost:5000>

```
Modo Desktop (Tkinter):
cd offline
python desktop_app.py
Executavel Compilado (Recomendado):
```

Clique duas vezes em `offline/dist/CyberbullyingDetector.exe`.

3.5 Configuracao

Variavel de Ambiente	Descricao	Padrao
---	---	---
`CYBERBULLYING_DB_PATH`	Caminho completo para a BD SQL	`data/cyberbullying.db`

Se a variavel nao for definida, o caminho e calculado automaticamente:

- ****Script****: diretorio raiz do projeto + `/data/cyberbullying.db`
- ****EXE****: procura recursivamente desde o diretorio do executavel ate a raiz

4. Guia de Utilizacao

4.1 Interface Web

Acesso: <http://localhost:5000>

****Barra lateral (sidebar):****

Botao	Visivel para	Descricao
---	---	---
Dashboard	Todos	Graficos interativos (classifi
Analise	Todos	Inserir texto e classificar
Alertas	Todos	Alertas pendentes (>= 50% conf
Analises	Todos	Historico de analises
Fontes	Todos	Gerir fontes Facebook
Dicionario	Todos	Gerir girias locais
Usuarios	Admin	Gerir utilizadores
Relatorio	Admin	Estatisticas e exportacao (CSV
Inicializar	Admin	Carregar dados iniciais
Importar Testes	Admin	Importar 10 comentarios de tes
Atualizar	Todos	Atualizar todos os dados
Minha Senha	Todos	Alterar propria senha
Sair	Todos	Terminar sessao

****Pagina de Analise:****

1. Digite o texto na caixa de texto
2. Clique "Analisar" ou pressione `Ctrl+Enter`
3. Veja a classificacao, confianca, palavras e girias detetadas

****Pagina de Alertas:****

- Lista comentarios com confianca >= 50% ainda nao resolvidos
- Botao para resolver (marcar como revisto)
- Botao para copiar alerta para area de transferencia (report ao Facebook)
- Notificacoes nativas do browser ao receber novo alerta via WebSocket

****Relatorio PDF:****

- Na pagina Relatorio, clique "Baixar PDF" para descarregar um relatorio completo em PDF
- O PDF e gerado com fpdf2 e inclui todas as estatisticas do sistema

4.2 Interface Desktop

****Abas disponiveis (admin):****

Aba	Descricao
---	---
Analise	Analisar texto com resultado d
Alertas	Alertas pendentes com auto-ref
Dicionario	Gerir girias e niveis
Fontes	Gerir fontes Facebook
Relatorio	Estatisticas e exportacao CSV/
Usuarios	Gerir equipa (admin apenas)

****Abas disponiveis (moderador):**** Todas exceto Relatorio e Usuarios.

****Atalhos de teclado:****

- `Ctrl+Enter` — analisar texto rapidamente

4.3 Gestao de Utilizadores (Admin)

****Criar utilizador:****

1. Aba Usuarios -> botao "Novo Usuario"
2. Preencher Nome, Email, Senha, Papel (admin/moderador)
3. Clicar "Salvar"

****Alterar senha de outro utilizador:****

1. Aba Usuarios -> selecionar utilizador na lista
2. Clicar "Alterar Senha"
3. Escolher novo utilizador e nova senha

****Alterar propria senha:****

- Web: sidebar -> "Minha Senha"
- Desktop: barra de status -> "Alterar Senha"

****Remover utilizador:****

1. Aba Usuarios -> selecionar utilizador
2. Clicar "Remover"

5. API REST

Todas as rotas (exceto `/login` e `/logout`) requerem autenticacao por sessao. Rotas ****Admin**** requerem `papel='admin'`.

5.1 Autenticacao

Metodo	Rota	Acesso	Descricao
---	---	---	---
GET	/login	Publico	Pagina de login
POST	/login	Publico	Autenticar (username, passw
GET	/logout	Publico	Terminar sessao

5.2 Dashboard e Estatisticas

Metodo	Rota	Acesso	Descricao
---	---	---	---
GET	/	Autenticado	Pagina principal (dashboard)
GET	/api/estatisticas	Autenticado	Estatisticas gerais do sistema
GET	/api/dashboard	Autenticado	Dados agregados para graficos
GET	/api/alertas	Autenticado	Alertas pendentes (confianca >
GET	/api/analises	Autenticado	Ultimas 100 analises com detal
POST	/api/analises/<id>/resolver	Autenticado	Marcar analise como resolvida

5.3 Comentarios

Metodo	Rota	Acesso	Descricao
---	---	---	---
GET	/api/comentarios	Autenticado	Ultimos 100 comentarios
POST	/api/comentarios	Autenticado	Adicionar comentario + analise

5.4 Detecao

Metodo	Rota	Acesso	Descricao
---	---	---	---
POST	/api/detectar	Autenticado	Analisar texto ({"texto": "..

5.5 Fontes

Metodo	Rota	Acesso	Descricao
--------	------	--------	-----------

---	---	---	---
GET	`/api/fontes`	Autenticado	Listar fontes Facebook
POST	`/api/fontes`	Autenticado	Adicionar fonte
DELETE	`/api/fontes/<id>`	Admin	Remover fonte

5.6 Dicionario

Metodo	Rota	Acesso	Descricao
---	---	---	---
GET	`/api/dicionario`	Autenticado	Listar girias
POST	`/api/dicionario`	Autenticado	Adicionar gria
DELETE	`/api/dicionario/<id>`	Admin	Remover gria

5.7 Utilizadores

Metodo	Rota	Acesso	Descricao
---	---	---	---
GET	`/api/usuarios`	Autenticado	Listar utilizadores
POST	`/api/usuarios`	Admin	Criar utilizador
DELETE	`/api/usuarios/<id>`	Admin	Remover utilizador
POST	`/api/alterar-senha`	Admin	Alterar senha de outro
POST	`/api/minha-senha`	Autenticado	Alterar propria senha

5.8 Utilitarios

Metodo	Rota	Acesso	Descricao
---	---	---	---
GET	`/api/relatorio`	Admin	Relatorio completo
GET	`/api/relatorio/pdf`	Admin	Download do relatorio em PDF
POST	`/api/inicializar`	Admin	Carregar dados iniciais
POST	`/api/importar-comentarios`	Admin	Importar 10 comentarios de tes

5.9 Exemplo de Analise

```
curl -X POST http://localhost:5000/api/detector \
-H "Content-Type: application/json" \
-d '{"texto": "Vou te matar seu idiota"}'
```

Resposta:

```
{
  "classificacao": "Critico",
  "confianca": 95.0,
  "girias": [],
  "palavras": [
    {"termo": "vou te matar", "nivel": "critico"},
    {"termo": "idiota", "nivel": "medio"}
  ],
  "critico": true,
  "nivel_geral": "critico"
}
```

6. Motor de Detecao

6.1 Arquitetura

O motor (`app/detector.py`) e um singleton baseado em regras (rule-based), partilhado entre web e desktop. Utiliza:

1. **Lista hardcoded**: 170+ palavras e expressoes ofensivas em portuges
2. **Dicionario local** (`data/local\_dictionary.json`): 80+ girias regionais angolanas carregadas dinamicamente
3. **Girias da BD** (`girias\_db`): girias adicionadas pelo utilizador

4. ****Classificador BERT** (`app/bert\_classifier.py`)**: complemento opcional

6.2 Sistema de Pontuacao

Fator	Pontos
---	---
Base (confianca inicial)	5.0
Por palavra/giria critica	+45
Por palavra/giria alta	+25
Por palavra/giria media	+15
Por palavra/giria baixa	+8
Bonus critico (se pelo menos 1	+35
MAIUSCULAS (>50% do texto em m	+10
Sinais de exclamacao/interroga	+2 cada (max. +10)
Maximo	**100**

6.3 Thresholds de Classificacao

Confianca	Classificacao	Alerta
---	---	---
>= 90%	Critico	Sim
>= 70%	Ofensivo	Sim
>= 50%	Suspeito	Sim
< 50%	Neutro	Nao

6.4 Nivel de Gravidade

O `nivel\_geral` e determinado pelo nivel mais alto entre todas as palavras e girias detetadas:

critico > alto > medio > baixo > neutro

6.5 Dicionario Local (JSON)

Ficheiro: `data/local\_dictionary.json`

Contem 80+ girias regionais de Angola, organizadas por nivel de gravidade:

```
{
  "girias": [
    {"termo": "kamba", "significado": "amigo", "nivel": "neutro"},
    {"termo": "mbora", "significado": "vamos embora", "nivel": "neutro"},
    {"termo": "safado", "significado": "descarado", "nivel": "alto"},
    {"termo": "muteta", "significado": "pessoa fraca", "nivel": "medio"}
  ]
}
```

As girias sao carregadas na BD na inicializacao (`/api/inicializar`), evitando INSERTs hardcoded.

7. Classificador BERT

7.1 Arquitetura

O `bert\_classifier.py` implementa um classificador baseado no modelo `distilbert-base-multilingual-cased` da Hugging Face. Este modelo foi escolhido por:

- Suportar multiplos idiomas (incluindo portugues e misturas com linguas angolanas)
- Ser mais leve que o BERT completo (menos parametros, inferencia mais rapida)
- Nao necessitar de fine-tuning no cenario atual

7.2 Funcionamento

1. O modelo e carregado `\_lazy\_` (apenas na primeira chamada a `classificar`)
2. O texto e convertido em embeddings usando a saida `pooler\_output` do modelo
3. A classificacao e feita por similaridade de cosseno entre o embedding do texto e embeddings de referencia pre-definidos (ofensivo, neutro)
4. Se o modelo nao carregar (ex.: sem torch/transformers instalados), faz fallback para Neutro com 0% de confianca

7.3 Integracao

O classificador BERT e chamado apos o motor rule-based. Se a confianca do BERT for superior a do motor rule-based, a classificacao final usa o resultado do BERT. Caso contrario, prevalece o resultado rule-based.

8. WebSockets e Notificacoes

8.1 Arquitetura

O sistema utiliza **Flask-SocketIO** para comunicacao em tempo real entre o servidor e os clientes web. O servidor emite eventos WebSocket sempre que um novo alerta e detetado ou resolvido.

8.2 Eventos

Evento	Direcao	Descricao
---	---	---
`novo_alerta`	servidor -> cliente	Emitido quando uma analise nov
`alerta_resolvido`	servidor -> cliente	Emitido quando um alerta e mar

8.3 Notificacoes no Browser

O cliente JavaScript (`main.js`) escuta eventos SocketIO e exibe notificacoes nativas usando a API `Notification` do browser:

```
socket.on('novo_alerta', function(alerta) {
  if (Notification.permission === 'granted') {
    new Notification('Alerta de Cyberbullying', {
      body: alerta.texto,
      icon: '/static/logo.png'
    });
  }
});
```

8.4 Async Mode

O SocketIO e configurado com `async\_mode='threading'` para compatibilidade com o EXE congelado (eventlet falha em ambientes PyInstaller). Em execucao normal com `python run.py`, usa eventlet para melhor desempenho.

9. Base de Dados

9.1 Esquema

Ficheiro: `data/cyberbullying.db`

Modo: WAL (Write-Ahead Logging) para concorrencia

Timeout: 10 segundos (multiplas conexoes)

Coluna	Tipo	Descricao
---	---	---
`id`	INTEGER PK	Identificador unico
`nome`	TEXT	Nome do utilizador
`email`	TEXT	Email
`senha`	TEXT	Hash bcrypt da palavra-passe
`papel`	TEXT	'admin' ou 'moderador'
`avatar`	TEXT	(reservado)

Coluna	Tipo	Descricao
---	---	---
`id`	INTEGER PK	Identificador unico
`url`	TEXT	URL da fonte (ex: grupo Facebo
`nome`	TEXT	Nome da fonte
`tipo`	TEXT	Tipo de fonte ('API', 'manual'
`ativo`	INTEGER	1=ativo, 0=inativo

Coluna	Tipo	Descricao
---	---	---
`id`	INTEGER PK	Identificador unico
`fonte_id`	INTEGER FK	Referencia a fonte
`texto`	TEXT	Conteudo do comentario
`autor`	TEXT	Autor do comentario
`data`	TEXT	Data/hora ISO 8601

Coluna	Tipo	Descricao
---	---	---
`id`	INTEGER PK	Identificador unico
`comentario_id`	INTEGER FK	Referencia ao comentario
`classificacao`	TEXT	Critico/Ofensivo/Suspeito/Neut
`confianca`	REAL	Porcentagem de confianca (0-10)
`gurias`	TEXT	Gurias detetadas (separadas po
`resolvido`	INTEGER	0=pendente, 1=resolvido
`data`	TEXT	Data/hora ISO 8601

Coluna	Tipo	Descricao
---	---	---
`id`	INTEGER PK	Identificador unico
`termo`	TEXT	Termo/giria
`significado`	TEXT	Significado
`tipo`	TEXT	'ofensivo' ou 'neutro'
`nivel`	TEXT	Nivel de gravidade (critico/al

9.2 Dados Iniciais (Seed)

****Utilizadores:****

Nome	Email	Senha (hash bcrypt)	Papel
---	---	---	---
Alberto Baptista	alberto@saurimo.ao	bcrypt de 240520	admin
Augusta Mulungia	augusta@saurimo.ao	bcrypt de 1234	moderador
Rafael Mussumari	rafael@saurimo.ao	bcrypt de 1234	moderador

****Fontes Facebook:**** Juventude Saurimo, Lunda-Sul Geral

****Gurias regionais:**** 80+ termos lidos de `data/local\_dictionary.json`

9.3 Migracao Automatica

Na primeira execucao, o sistema:

1. Cria as tabelas se nao existem
2. Adiciona colunas em falta (migracao segura com tratamento de erros)
3. Migra senhas em texto plano para bcrypt automaticamente
4. Atualiza o utilizador admin para "Alberto Baptista" / "240520"
5. Remove duplicados do utilizador admin antigo

10. Gestao de Utilizadores e Permissoes

10.1 Papeis (Roles)

Papel	Acesso
---	---
`admin`	Todas as funcionalidades, incl
`moderador`	Analise, alertas, analises, fo

10.2 Controlo de Acesso

Web (JavaScript): Elementos com atributo `data-admin` sao ocultados para moderadores.

Web (Python decorators):

```
def admin_required(f):
```

```
    @wraps(f)
```

```
    def decorated(*args, **kwargs):
```

```
        if session.get('papel') != 'admin':
```

```
            return jsonify({'erro': 'Acesso negado'}), 403
```

```
        return f(*args, **kwargs)
```

```
    return decorated
```

Desktop: Abas de Relatorio e Usuarios sao adicionadas apenas se `self.papel == 'admin'`.

10.3 Seguranca

- **Senhas com bcrypt:** hasheadas com custo 12; migracao automatica de texto plano existente
- **Offline-first:** o sistema funciona 100% offline (localhost)
- **Servidor web:** bind a `0.0.0.0:5000` — acessivel apenas na rede local
- **Chave secreta:** definida no codigo (`SECRET\_KEY`) — adequado para ambiente academico controlado

11. Testes Automatizados

11.1 Estrutura

Os testes estao em `tests/` e utilizam pytest com uma fixture de BD temporaria (`conftest.py`):

```
python -m pytest tests/ -v
```

11.2 Testes do Motor de Detecao (`test\_detector.py`)

Teste	Descricao
---	---
`test_texto_neutro`	Texto normal -> Neutro com < 5
`test_texto_critico`	Ameaca direta -> Critico com >
`test_texto_ofensivo`	Palavrao grave -> Ofensivo com
`test_texto_suspeito`	Palavra media -> Suspeito com
`test_girias_detetadas`	Girias regionais sao detetadas
`test_maiusculas`	Texto em MAIUSCULAS ganha bonu

11.3 Testes da API (`test\_api.py`)

Teste	Descricao
---	---
`test_login`	Login valido retorna 200 e red
`test_detectar`	API de detecao retorna classif
`test_alertas`	Endpoint de alertas retorna li
`test_fontes`	CRUD de fontes funciona
`test_dicionario`	CRUD de girias funciona
`test_inicializar`	Seed data carrega sem erros

11.4 Testes de Autenticacao (`test\_auth.py`)

Teste	Descricao
---	---
`test_login_valido`	Credenciais corretas -> sessao
`test_login_invalido`	Senha errada -> 401
`test_logout`	Logout limpa sessao e redireci

12. Empacotamento (EXE)

12.1 Construir o Executavel (--onefile)

```
$env:TCL_LIBRARY = "C:\caminho\para\python\tcl\tcl8.6"
$env:TK_LIBRARY = "C:\caminho\para\python\tcl\tk8.6"
cd offline
py -m PyInstaller --onefile --noconsole --name CyberbullyingDetector ^
--add-data "..\app;app" ^
--add-data "..\data\local_dictionary.json;data" ^
--add-data "logo.png;" ^
--hidden-import flask_cors ^
--hidden-import flask_socketio ^
--hidden-import flask_sqlalchemy ^
--hidden-import bcrypt ^
--hidden-import pdf2 ^
--hidden-import eventlet ^
--hidden-import numpy ^
--hidden-import requests ^
--collect-all flask_sqlalchemy ^
desktop_app.py
**Tempo de compilacao:** ~1-2 minutos (--onefile), ~35 segundos (--onedir)
```

12.2 Hidden Imports

Os seguintes modulos sao explicitamente incluidos para evitar erros em tempo de execucao no EXE compilado:

```
- `flask_cors`, `flask_socketio`, `flask_sqlalchemy`
- `bcrypt`, `pdf2`, `eventlet`, `numpy`, `requests`
- `sqlalchemy`, `jinja2`, `markupsafe`, `itsdangerous`
- `werkzeug`, `click`, `flask`
- `tkinter`, `tkinter.ttk`, `tkinter.messagebox`, `tkinter.filedialog`
```

12.3 Variaveis TCL_LIBRARY / TK_LIBRARY

Estas variaveis de ambiente sao **obrigatorias** no Python 3.14 + PyInstaller para evitar o erro:

FileNotFoundError: Tcl data directory "..._internal\tcl_data" not found.

Apontam para os diretorios Tcl/Tk dentro da instalacao Python.

12.4 Executavel Compilado

Localizacao: `offline/dist/CyberbullyingDetector.exe` (35 MB)

Modo: `--noconsole` (sem janela de terminal)

Na primeira execucao, cria a BD partilhada no directorio do projeto (procura recursivamente por `data/cyberbullying.db`).

13. Manutencao e Resolucao de Problemas

13.1 Log de Diagnostico

O sistema escreve para `data/debug.log`:

- Estado da BD antes/depois da migracao
- Tentativas de login (utilizador, senha, resultado)
- Erros de conexao a BD
- Diagnostico automatico ao iniciar

13.2 Problemas Comuns

Problema	Causa Provavel	Solucao
---	---	---
"Usuario ou senha invalidos"	BD corrompida ou migracao inco	Apagar `data/cyberbullying.db`
"Database is locked"	Concorrencia entre web e deskt	Aguardar ou reiniciar. WAL mod
Servidor web nao inicia	Porta 5000 ocupada	Fechar outras instancias do FI
EXE nao abre	TCL_LIBRARY nao definido na co	Recompilar com as variaveis de
Login falha apos logout (deskt)	Thread do servidor web ainda a	Fechar completamente o EXE e r
Invalid async_mode specified	eventlet incompativel com EXE	Usar `async_mode='threading'`
Modulo nao encontrado no EXE	Hidden import em falta	Adicionar `--hidden-import` ao

13.3 Reset Completo

Apagar base de dados

```
rm data/cyberbullying.db
```

Apagar log

```
rm data/debug.log
```

Reiniciar

```
python run.py
```

Na primeira execucao, o sistema recia a BD e insere os dados iniciais automaticamente.

13.4 Verificacao de Integridade

No desktop, o diagnostico automatico corre ao iniciar e regista no log:

- Caminho da BD
- Se o ficheiro existe
- Numero de analises, comentarios e alertas
- Ultimas 5 analises

14. Credenciais

Sistema	Utilizador	Senha	Papel
---	---	---	---
Admin	Alberto Baptista	240520	admin
Moderador	Augusta Mulungia	1234	moderador
Moderador	Rafael Mussumari	1234	moderador

As senhas sao armazenadas com hash bcrypt. Na primeira execucao, senhas em texto plano sao migradas automaticamente.

****Repositorio GitHub:**** <https://github.com/24Anselmo/Cyber123>

15. Equipa de Desenvolvimento

****Disciplina:**** PP1 — Engenharia de Software I + Sistemas Operacionais

Membro	Papel no Projeto
---	---
Alberto Baptista	Desenvolvedor principal
Augusta Mulungia	Documentacao e testes
Rafael Mussumari	Interface e validacao

****Instituicao:**** Instituto Superior Politecnico da Lunda-Sul (IPLS)

****Localizacao:**** Saurimo, Angola

****Ano Letivo:**** 2026

Documentacao gerada em Junho de 2026. Para mais informacoes, contactar a equipa de desenvolvimento ou consultar o repositorio GitHub.